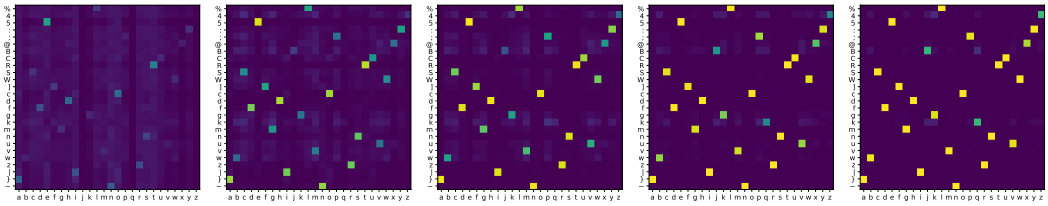


Figure 5: Predicted cipher key for windows of size 50, at indices 0, 50, 100, 200, and 400. Generated using temperature of $T=1$.

Finally, we aggregate over the last window of the 1000 sequences to recover a full cipher key, in case certain letters never appear in the last window of certain sequences. We again recover a full cipher key via majority vote. In Figure 4b, we show the highly accurate predicted cipher key recovered from ciphertext produced using the example ground truth cipher key in Figure 4a.

To perform a more detailed analysis showing the Bayesian deciphering process of the lexinvariant model, we use the logits of the probe to recover the predicted distribution of the cipher key $\mathcal{P}(\pi|x_1,\ldots,x_{n-1})$. Instead of taking the majority vote of the predicted decipher tokens in the window, we take the mean of logits predicted for each ciphered token. This essentially gives a locally averaged predicted distribution of cipher key matrices. Specifically, the cipher key matrices are generated across windows of 50 characters, and the probabilities are averaged over 1000 input sequences encoded using the same ground truth cipher. As shown in Figure 5, the predicted distribution of cipher key matrix becomes sharper as the prefix becomes longer.

3.4 In-context Bayesian Deciphering Examples

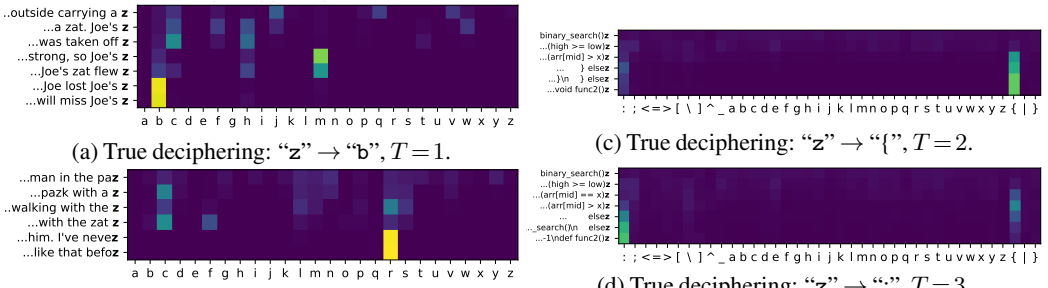
Here, we show several qualitative examples of in-context Bayesian deciphering. We first show how the lexinvariant LM maintains uncertainty over possible lexical permutations while iteratively updating them at each index, using examples from a character-level lexinvariant model. Then, we also show an example of semantic in-context deciphering with a 32K vocabulary lexinvariant model, where the meaning of a novel word is inferred relative to common words in-context.

3.4.1 Uncertainty over Lexical Permutations

In Figure 6a, we input the following ciphered sequence to the frozen character-level lexinvariant LM with the probe: "`I saw lots of people in town today, walking and talking around me. I greeted my friend Alice and my classmate Alex. I saw a guy, Joe, walking outside carrying a zat. Joe's zat was taken off zy wind. Today's wind was strong, so Joe's zat flew zackward. Joe lost Joe's zat for good. Joe will miss Joe's zat.`" For each instance of `z` in the sequence, we display the predicted deciphering of that instance as a row of probabilities across non-cipher letters `a`-`z`.

The lexinvariant model starts off assuming uniform probability for all possible lexical permutations π. After seeing more and more text, the lexinvariant model quickly realizes that `z` only has a few main plausible decipherings (`b`, `h`, `c`, `m`). Eventually, the lexinvariant model is able to narrow the possibilities down to `z` maps to `b` near the end of the sequence. The predicted probabilities shift with the seen context accordingly, demonstrating an example of how the predicted cipher key is iteratively updated at each index.

Figure 6b shows another example with a similar set up, but with text: "`I saw a man in the pazk with a zat. The man was walking with the zat zight beside him. I've nevez`



(a) True deciphering: "`z`" → "`b`", $T=1$.

(c) True deciphering: "`z`" → "{", $T=2$.

(b) True deciphering: "`z`" → "`r`", $T=1$.

(d) True deciphering: "`z`" → ":", $T=3$.

Figure 6: Probe predictions for deciphering "`z`" at each occurrence of "`z`" in context.

`seen anything like that befoze.`" While context initially suggests that `z` may be deciphered as `c`, it becomes clear that `z` must correspond to `r` after the appearance of "`right`". The disambiguation is reflected in the depicted probabilities.

In Figure 6c and 6d, we show two deciphering examples over code. We consider two code examples in which it is initially ambiguous whether the character `z` deciphers to `:` or `{`. The ambiguity is eventually resolved by the use of Python-like or Java-like syntax.

3.4.2 Semantic Deciphering

In addition to character-level deciphering, we show examples of semantic deciphering with the larger vocabulary of 32k. Although the lexinvariant LM could not possibly figure out the true lexical permutation among 32k tokens using a small 512 context, it is possible to construct a simple context that repetitively uses simple words so that these words are easier to decipher. Then the lexinvariant LM can decipher the approximate semantics of the rare symbols relative to other easier-to-decipher words.

One example is the following: given the prompt `'crash!' 'aaah!' i looked up from my cup of coffee. 'crash!' - that was the cafe window. and 'aaah!'` [... more text...] `what one here is a drink` ↵ `- restaurants` ↵ `- music` ↵ `- coffee` ↵ `- father` ↵ `the one here that drink is`, where the word `coffee`, `music`, and `father` all only appear once before the question and `restaurants` appeared 4 times, the model is able to correctly answer that `coffee` is drinkable. See the full example in the appendix.

3.5 Synthetic Reasoning Tasks

As discussed in the introduction, lexical flexibility is correlated with in-context reasoning performance, as demonstrated by existing large LMs. Thus, we study whether the lexinvariant model also learns in-context reasoning capabilities through the challenging lexinvariant training.

Specifically, we measure the performance of lexinvariant models over two pure in-context symbol manipulation tasks: LookUp, where the task is to predict the next token based on the given lookup table, e.g. `A->2`↵`C->4`↵`G->5`↵`C->` (should predict `4` here); and Permutation, where the task is to permute an arbitrary subsequence of the given sequence the same way as in the given few demonstrations, e.g. `A 2 C->C A`↵`4 1 D->` (should predict `D 4` here). In each of the tasks, the symbols are randomly sampled from the vocabulary so that we measure the pure reasoning ability independent from any knowledge of specific words. We measure the model performance in terms of generated token accuracy over 1000 examples. The results are shown in Table 1. As shown in the table, the lexinvariant models achieve drastically higher accuracy, with an average of 4X improvement.

Table 1: Accuracy over synthetic reasoning tasks.

Dataset	Vocab	LookUp Acc		Permutation Acc	
		Standard	LI	Standard	LI
Pile	char	48.50	91.80	27.66	59.35
	32k	21.45	92.10	22.84	55.63
Wiki-40B	char	38.25	59.70	20.77	60.51
	32k	8.75	59.35	9.94	50.91
Github	char	42.40	86.65	21.03	71.59
	32k	4.25	80.20	8.59	67.39

3.6 Regularizing Language Models with Lexinvariance

Although lexinvariant LM has various interesting properties , it is not suitable for practical tasks since it would require the context to be extremely long so that all required words and knowledge are defined in the context. Here, we explore how to construct more practical semi-lexinvariant LMs that maintain some properties of lexinvariant LMs via regularization. We emphasize that this exploration is intended to be illustrative rather than directly improving state-of-the-art.

Instead of using random Gaussian embedding matrices in place of a learned embedding matrix entirely, we can use random embeddings for only some of the tokens in each sequence, while others use the learned embedding. This means that the learned LM assumes that certain tokens have stable meanings

but not others, which can be seen as a form of regularization towards lexinvariance. Specifically, we randomly select tokens to randomize based on a Bernoulli distribution, which can essentially be seen as a form of dropout on token embeddings. On the BIG-bench tasks, we found that a model with dropout rate $p = 0.2$ for randomization was 25% more likely to improve performance than to harm performance when evaluated with three shots, relative to a comparably-sized LM, with improvements especially over retrieval type of tasks. See full details in the Appendix G.

More broadly, this regularization view could potentially bring the benefit of lexinvariant LMs to practical applications. For example, the regularization could improve 1) the robustness of LMs by making them less sensitive to adversarial attacks or noise in the input data, 2) generalization across different languages or domains by being less tied to specific lexical items and more prone to learn the shared language structure, and 3) reasoning over more realistic tasks as we have started to explore with BIG-Bench. These areas are promising directions for future work to explore.

4 Related Work

4.1 Symbol Grounding

Beyond a modeling choice, the main question of our paper (that being whether an LM can learn language without a stable token representation) is also analogous to the symbol grounding problem: Can meaning be acquired when symbols are not even grounded stably, i.e. they can be mapped to completely random meanings in different sequences? It has long been argued by the symbol grounding literature that symbolic representations must be grounded bottom-up in nonsymbolic representations [11], with famous arguments like Searle's Chinese room. And this leads to an ongoing debate on whether LMs can learn meaning purely from large amounts of text, without grounding to any real-world objects [4]. Conceptually, lexinvariant LM is one step further away from physical grounding. We show that they can still infer the meaning of symbols based on lexical structures within the context.

4.2 Byte-level T5

There is existing work on absorbing tokenization completely into part of language modeling by using extremely small tokens, such as Byte-level T5 [25]. In the extreme, such a model would become closer and closer to lexinvariant LM, since bytes or bits have almost no stable meaning, so their embeddings are likely not used for prediction. In this paper, we study general lexinvariant LMs with the lexinvariant property baked in and without requiring specific tokenizers.

4.3 Group invariances and Data augmentation

Our implementation of lexinvariant LMs can be seen as performing a form of very aggressive data augmentation, where we randomize the identity of each token in each sequence. From this perspective, it is somewhat similar to the data recombination in [14, 2] and augmentation of named entities in [20], where certain parts of the sentence are swapped with other words while still maintaining the original grammatical structure. In contrast to these augmentations, the training for our lexinvariant LMs completely swaps out all parts of the input text.

4.4 Deciphering Substitution Cipher using LMs

In general, solving substitution ciphers, where the cipher key is a permutation of the original alphabet, is a NP-hard problem when only having access to LMs that can assign probabilities to sequences [18]. There have been several works focusing on solving substitution ciphers using LMs, including approaches from searching over the permutation space guided by LMs' scores [12] to training a seq-to-seq model directly to perform deciphering as translation [3]. Although our work does not focus specifically on the task of deciphering substitution ciphers, we show that our lexinvariant model can efficiently perform in-context deciphering as a byproduct of language modeling.

4.5 Reasoning

It has been shown that large language models acquire surprising in-context reasoning capabilities [6, 15, 23]. Many of them are related to lexical flexibility through training for purely next-token prediction, such as modified arithmetic, data reformatting, and redefining single word etc. However, LLMs also memorize an enormous amount of knowledge along the way, which is often unnecessary.